

PyTorch Distributed Training

Complete Reference Notes

Parallelism Strategies · DDP · FSDP · Mixed Precision · Optimisations · Fine-tuning

Data Parallelism · Tensor Parallelism · Pipeline Parallelism · Expert Parallelism · 3D / 4D Parallelism
· FSDP · DDP · torchrun · DeviceMesh · Gradient Checkpointing · Mixed Precision · ZeRO ·
Multi-GPU Fine-tuning

Based on official PyTorch documentation, tutorials, cookbooks & research papers

1. Foundations of Distributed Training

1.1 Why Distributed Training?

Modern deep learning models — from GPT-class language models to large vision transformers — have crossed the billion-parameter threshold. Training such models on a single GPU is impractical for three reasons: memory capacity, compute throughput, and time. Distributed training addresses all three by spreading work across multiple processing units.

Core Motivations

- **Memory:** A single A100 80 GB GPU cannot hold a 70B-parameter model with fp32 weights (~280 GB), gradients, and optimizer states.
- **Speed:** Parallelising across 8× GPUs can yield near-linear training speedup for well-tuned workloads.
- **Scale:** Multi-node clusters (100s–1000s of GPUs) are required to train frontier models in reasonable wall-clock time.
- **Fault-tolerance:** Distributed checkpointing and elastic training allow jobs to survive node failures.

1.2 Key Concepts & Terminology

Term	Meaning
rank	Unique integer ID of a process within the distributed group (0 to world_size-1)
world_size	Total number of processes participating in training
local_rank	Rank of the process on the current machine (used to assign GPU)
node / host	A physical or virtual machine that can contain 1–N GPUs
process group	A subset of processes that can communicate collectively
backend	Communication library: NCCL (GPU-GPU), Gloo (CPU/fallback), MPI
collective op	Communication ops like AllReduce, AllGather, ReduceScatter, Broadcast, Barrier
all-reduce	Aggregates (sum/avg) a tensor across all ranks and distributes result back to all
all-gather	Each rank contributes a tensor shard; all ranks receive the full tensor
reduce-scatter	Reduces a tensor across ranks, then distributes non-overlapping shards
DeviceMesh	Abstraction that arranges devices in an N-D grid for composable parallelism
DTensor	Distributed Tensor — a tensor that is sharded/replicated across a DeviceMesh
shard	A partition of a tensor or model parameters held by one process

1.3 Memory Breakdown in Training

Understanding where GPU memory goes is essential to choosing the right parallelism strategy.

Category	fp32 (bytes per param)	Mixed Prec. (approx.)
Model weights	4 bytes	2 bytes (fp16/bf16)
Gradients	4 bytes	2 bytes (fp16/bf16)
Optimizer states (Adam)	8 bytes (m + v)	8 bytes (kept in fp32)
Activations	Proportional to batch × seq × hidden	Halved with fp16
Total (no sharding)	~16× params (fp32 Adam)	~6× params (mixed prec.)

1.4 Parallelism Taxonomy

PyTorch and the broader ecosystem support five main dimensions of parallelism. They are orthogonal and can be composed.

Type	What is Sharded	Communication	When to Use
Data Parallelism (DP)	Input data batches	AllReduce of gradients	Model fits 1 GPU; scale throughput
Tensor Parallelism (TP)	Individual weight tensors (intra-layer)	AllReduce / AllGather within layer	Single layer too large; >256 GPUs
Pipeline Parallelism (PP)	Consecutive model layers	Fwd/bwd activations at stage boundaries	Model too tall; > 1 node
Expert Parallelism (EP)	MoE expert networks	AllToAll tokens dispatch/combine	MoE architectures
Sequence Parallelism (SP)	Input sequence tokens	AllGather / ReduceScatter	Long-context training alongside TP

1.5 Communication Backends

Backend	Characteristics
NCCL	Optimised for NVIDIA GPUs; uses NVLink & InfiniBand; required for GPU-GPU communication
Gloo	CPU-based; fallback for CPU tensors or Windows; no NVLink benefit
MPI	Flexible; good for HPC clusters; requires manual MPI installation
UCC	Experimental; unified collective communication for heterogeneous hardware

2. Data Parallelism

2.1 DataParallel (DP) — Legacy Single-Machine

`torch.nn.DataParallel` (DP) is the original single-process, multi-threaded wrapper. It splits each mini-batch across GPUs, runs forward passes in parallel, gathers outputs to GPU 0, computes loss on GPU 0, scatters gradients back, and reduces. It is simple but suffers from GIL contention, memory imbalance on GPU 0, and single-machine restriction.

```
import torch
import torch.nn as nn

model = MyModel()
# Wrap model — auto-uses all visible GPUs, or specify device_ids
model = nn.DataParallel(model, device_ids=[0, 1, 2, 3])
model = model.cuda()

# Standard training loop — no other changes needed
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
for inputs, targets in dataloader:
    inputs, targets = inputs.cuda(), targets.cuda()
    outputs = model(inputs)          # inputs auto-split across GPUs
    loss = criterion(outputs, targets)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# WARNING: DP has a known memory imbalance — GPU 0 stores all gradients.
# For production use, always prefer DDP.
```

DataParallel Limitations

- Single-process → Python GIL limits true parallelism across threads
- GPU 0 bottleneck: gathers all outputs, computes loss, holds extra gradient copies
- No multi-node support
- Cannot be combined with model parallelism properly
- Deprecated for most production workloads — use DDP instead

2.2 DistributedDataParallel (DDP) — Recommended

`DistributedDataParallel` (DDP) in `torch.nn.parallel` assigns one process per GPU (or group of GPUs in model-parallel scenarios). Processes communicate purely through collective operations — no parameter-server bottleneck. DDP is the default starting point for multi-GPU training in PyTorch.

2.2.1 DDP Internals

DDP registers autograd hooks on each parameter during the backward pass. When gradients are ready, the hook triggers an asynchronous AllReduce over the gradient tensor, averaging it across all ranks. DDP overlaps this communication with the computation of remaining gradients (gradient bucketing), effectively hiding much of the communication latency.

DDP Phase	What Happens
Constructor	Model state broadcast from rank 0 to all other ranks; autograd hooks registered; gradient buckets allocated
Forward pass	Each process runs forward on its local mini-batch; no communication
Backward pass	Autograd hooks fire as gradient tensors are produced; gradients accumulated into buckets
AllReduce	Each full bucket triggers an async AllReduce; gradients averaged across all ranks
optimizer.step()	Identical averaged gradients applied; parameters stay in sync automatically

2.2.2 Complete DDP Training Script

```
# train_ddp.py - launch with: torchrun --nproc_per_node=4 train_ddp.py
import os
import torch
import torch.nn as nn
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, DistributedSampler
from torch.utils.data import TensorDataset

# — 1. Setup distributed environment —
def setup():
    dist.init_process_group(backend="nccl")
    torch.cuda.set_device(int(os.environ['LOCAL_RANK']))

def cleanup():
    dist.destroy_process_group()

# — 2. Define a simple model —
class ToyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1024, 512), nn.ReLU(),
            nn.Linear(512, 128), nn.ReLU(),
            nn.Linear(128, 10),
        )
    def forward(self, x): return self.net(x)

# — 3. Training function —
def train():
```

```

setup()
rank      = dist.get_rank()
local_rank = int(os.environ['LOCAL_RANK'])
device    = torch.device(f'cuda:{local_rank}')

# Model on GPU, wrapped with DDP
model = ToyModel().to(device)
ddp_model = DDP(model, device_ids=[local_rank])

# Dataset with DistributedSampler (ensures non-overlapping data per rank)
dataset = TensorDataset(
    torch.randn(1000, 1024), torch.randint(0, 10, (1000,))
)
sampler = DistributedSampler(dataset, num_replicas=dist.get_world_size(),
                              rank=rank, shuffle=True)
loader  = DataLoader(dataset, batch_size=32, sampler=sampler,
                     num_workers=4, pin_memory=True)

criterion = nn.CrossEntropyLoss().to(device)
optimizer = torch.optim.AdamW(ddp_model.parameters(), lr=1e-3)

for epoch in range(10):
    sampler.set_epoch(epoch) # CRITICAL: shuffles differently per epoch
    ddp_model.train()
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        loss = criterion(ddp_model(x), y)
        loss.backward() # gradient AllReduce happens here
        optimizer.step()

    if rank == 0: # only rank 0 should log/save
        print(f'Epoch {epoch} - loss: {loss.item():.4f}')

# Save checkpoint from rank 0
if rank == 0:
    torch.save(ddp_model.module.state_dict(), 'checkpoint.pt')

cleanup()

if __name__ == '__main__':
    train()

```

2.2.3 Launching DDP with torchrun

```

# — Single node, 4 GPUs —
torchrun --standalone --nproc_per_node=4 train_ddp.py

# — Multi-node: 2 nodes, 8 GPUs each —
# On node 0 (master):
torchrun \
    --nproc_per_node=8 \
    --nnodes=2 \

```

```

--node_rank=0 \
--master_addr=<NODE0_IP> \
--master_port=29500 \
train_ddp.py

# On node 1:
torchrun \
  --nproc_per_node=8 \
  --nnodes=2 \
  --node_rank=1 \
  --master_addr=<NODE0_IP> \
  --master_port=29500 \
  train_ddp.py

# — Environment variables set by torchrun —
# MASTER_ADDR — IP/hostname of rank-0 process
# MASTER_PORT — port for rendezvous
# RANK — global rank of this process
# LOCAL_RANK — rank on this machine (used to select GPU)
# WORLD_SIZE — total number of processes

```

2.2.4 DDP Key Parameters & Tips

Parameter / Pattern	Guidance
find_unused_parameters=True	Set if model has conditionally unused params (e.g., dynamic graphs). Adds overhead — avoid if possible.
static_graph=True	Set when graph is identical every iteration. Enables reentrant backward, activation checkpointing with unused params.
bucket_cap_mb	Gradient bucket size (default 25 MB). Larger buckets = fewer AllReduce calls = higher throughput on fast interconnects.
gradient_as_bucket_view=True	Reduces memory by making gradients a view of the bucket — avoids extra copy.
sampler.set_epoch(epoch)	MUST be called each epoch when using DistributedSampler to ensure different shuffling per epoch.
ddp_model.module	Access the underlying unwrapped model for saving checkpoints.
no_sync() context	Suppress gradient sync for gradient accumulation steps — only sync on the last micro-step.

2.2.5 Gradient Accumulation with DDP

```

accumulation_steps = 4
optimizer.zero_grad()

for i, (x, y) in enumerate(loader):
    x, y = x.to(device), y.to(device)

    # Suppress gradient sync for all steps except the last

```



```
if (i + 1) % accumulation_steps != 0:
    with ddp_model.no_sync(): # avoids AllReduce on micro-steps
        loss = criterion(ddp_model(x), y) / accumulation_steps
        loss.backward()
else:
    loss = criterion(ddp_model(x), y) / accumulation_steps
    loss.backward()          # AllReduce fires here
    optimizer.step()
    optimizer.zero_grad()
```

3. Fully Sharded Data Parallel (FSDP)

FSDP (`torch.distributed.fsdp`) implements ZeRO-3 sharding natively in PyTorch. Instead of replicating the full model on every GPU as DDP does, FSDP shards model parameters, gradients, and optimizer states across all data-parallel workers. During the forward and backward passes, parameters are gathered on demand with AllGather, used locally, then freed — dramatically reducing per-GPU memory footprint.

3.1 FSDP vs. DDP Memory Comparison

Component	DDP (per GPU)	FSDP ZeRO-3 (per GPU)	Savings (N GPUs)
Model params	Full copy	1/N shard	N× reduction
Gradients	Full copy	1/N shard	N× reduction
Optimizer states	Full copy (Adam: 2× params)	1/N shard	N× reduction
Total (approx.)	~16 bytes/param (fp32 Adam)	~16/N + comm overhead	Near-linear scaling

3.2 FSDP Sharding Strategies

Strategy	Description
FULL_SHARD (ZeRO-3)	Shards params, grads, and optimizer states. Maximum memory savings. Default for large models.
SHARD_GRAD_OP (ZeRO-2)	Shards only grads and optimizer states; keeps full params during forward. Less communication.
NO_SHARD	No sharding — equivalent to DDP. Use for debugging or when memory is not the bottleneck.
HYBRID_SHARD	Full shard within each node; replicate across nodes. Reduces cross-node communication.
_HYBRID_SHARD_ZERO2	ZeRO-2 intra-node; replicate across nodes.

3.3 FSDP Forward/Backward Execution Flow

FSDP Execution per Layer
Forward Pass:
1. AllGather — All ranks contribute their shard; each rank assembles full-layer params
2. Compute — Local forward computation with unsharded params
3. Discard — Non-owned param shards are freed to reclaim memory
Backward Pass:

1. AllGather — Params re-gathered for gradient computation
2. Compute — Local backward; gradients computed
3. ReduceScatter — Gradients averaged AND sharded in a single op
4. Discard — Non-owned param shards freed again

Optimizer Step:

Each rank updates only its own parameter shard using its own gradient shard

3.4 FSDP1 — Full Implementation Example

```
# train_fsd.py - launch with: torchrun --nproc_per_node=8 train_fsd.py
import os, functools
import torch
import torch.nn as nn
import torch.distributed as dist
from torch.distributed.fsdpy import FullyShardedDataParallel as FSDP
from torch.distributed.fsdpy import ShardingStrategy, MixedPrecision
from torch.distributed.fsdpy.wrap import transformer_auto_wrap_policy
from torch.utils.data import DataLoader, DistributedSampler, TensorDataset
from transformers import GPT2Config, GPT2Model

def setup():
    dist.init_process_group(backend="nccl")
    torch.cuda.set_device(int(os.environ['LOCAL_RANK']))

def cleanup():
    dist.destroy_process_group()

# — Mixed-precision policy (bf16 for compute, fp32 for reduction) —
mp_policy = MixedPrecision(
    param_dtype=torch.bfloat16,
    reduce_dtype=torch.float32,
    buffer_dtype=torch.bfloat16,
)

def train():
    setup()
    local_rank = int(os.environ['LOCAL_RANK'])
    device = torch.device(f'cuda:{local_rank}')
    rank = dist.get_rank()

    # — Build model on CPU first, then wrap with FSDP —
    config = GPT2Config(n_layer=24, n_head=16, n_embd=1024)
    model = GPT2Model(config) # stays on CPU at this point

    # — Auto-wrap policy: wrap each GPT2Block independently —
    from transformers.models.gpt2.modeling_gpt2 import GPT2Block
    wrap_policy = functools.partial(
        transformer_auto_wrap_policy,
        transformer_layer_cls={GPT2Block},
    )
```

```

fsdp_model = FSDP(
    model,
    auto_wrap_policy=wrap_policy,
    mixed_precision=mp_policy,
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    device_id=torch.cuda.current_device(),
    use_orig_params=True,          # required for gradient clipping & some
optimizers
)

optimizer = torch.optim.AdamW(fsdp_model.parameters(), lr=1e-4,
                               weight_decay=0.1)

dataset = TensorDataset(
    torch.randint(0, 50257, (512, 1024)), # token IDs
)
sampler = DistributedSampler(dataset)
loader = DataLoader(dataset, batch_size=4, sampler=sampler,
                    num_workers=4, pin_memory=True)

for epoch in range(3):
    sampler.set_epoch(epoch)
    fsdp_model.train()
    for (tokens,) in loader:
        tokens = tokens.to(device)
        optimizer.zero_grad()
        out = fsdp_model(input_ids=tokens)
        # Dummy LM loss
        loss = out.last_hidden_state.mean()
        loss.backward()
        # Gradient clipping - use FSDP's clip_grad_norm_
        fsdp_model.clip_grad_norm_(max_norm=1.0)
        optimizer.step()

    if rank == 0:
        print(f'Epoch {epoch}: loss={loss.item():.4f}')

# — Save full checkpoint (gathers all shards to CPU on rank 0) —
from torch.distributed.fsdp import FullStateDictConfig, StateDictType
save_policy = FullStateDictConfig(offload_to_cpu=True, rank0_only=True)
with FSDP.state_dict_type(fsdp_model, StateDictType.FULL_STATE_DICT,
save_policy):
    cpu_state = fsdp_model.state_dict()
    if rank == 0:
        torch.save(cpu_state, 'fsdp_checkpoint.pt')
        print('Checkpoint saved.')

cleanup()

if __name__ == '__main__':
    train()

```

3.5 FSDP2 — Next-Generation API (PyTorch 2.4+)

FSDP2 (`torch.distributed.fsdp.fully_shard`) is the successor to FSDP1. It uses per-parameter DTensor-based dim-0 sharding instead of flat-parameter sharding, providing more intuitive semantics, simpler reasoning about memory, and better composability with Tensor Parallelism.

```
# FSDP2 - per-parameter sharding using fully_shard()
from torch.distributed.fsdp import fully_shard, MixedPrecisionPolicy
from torch.distributed.device_mesh import init_device_mesh

# Create a 1-D device mesh over all GPUs
mesh = init_device_mesh('cuda', (8,))

model = MyTransformer()

# Apply FSDP2 to each transformer block first (bottom-up)
for block in model.transformer_blocks:
    fully_shard(block, mesh=mesh)

# Then apply to the whole model
fully_shard(model, mesh=mesh)

# Mixed precision policy
mp = MixedPrecisionPolicy(param_dtype=torch.bfloat16,
                          reduce_dtype=torch.float32)
fully_shard(model, mesh=mesh, mp_policy=mp)

# Training proceeds identically to DDP
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
for x, y in dataloader:
    loss = criterion(model(x), y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

3.6 Activation Checkpointing with FSDP

```
from torch.distributed.fsdp.wrap import checkpoint_wrapper
from torch.distributed.fsdp import CheckpointImpl

# Option 1: Wrap individual modules
from torch.utils.checkpoint import checkpoint_wrapper as torch_ckpt

# Option 2: Apply FSDP checkpointing utility
from torch.distributed.fsdp._wrap_utils import _apply_to_modules
import torch.distributed.algorithms._checkpoint.checkpoint_wrapper as cw

# Recommended: use apply_activation_checkpointing
from torch.distributed.algorithms._checkpoint.checkpoint_wrapper import (
    apply_activation_checkpointing, CheckpointWrapper, CheckpointImpl
)
```

```

from transformers.models.gpt2.modeling_gpt2 import GPT2Block
check_fn = lambda submodule: isinstance(submodule, GPT2Block)

apply_activation_checkpointing(
    fsdp_model,
    checkpoint_wrapper_fn=CheckpointWrapper,
    check_fn=check_fn,
)
# This re-computes activations during backward instead of storing them,
# trading extra compute for large memory savings.

```

3.7 Checkpointing Strategies in FSDP

Strategy	Description
FULL_STATE_DICT	Gathers all shards to CPU on rank 0. Saves a standard checkpoint. Suitable for portability.
SHARDED_STATE_DICT	Each rank saves its own shard independently. Faster I/O, requires same world_size to load.
LOCAL_STATE_DICT	Each rank saves its flat-parameter shard. Used for resuming on identical topology.
Distributed Checkpoint (dcp)	torch.distributed.checkpoint — parallel save/load, shard-independent, recommended for large models.

```

# Distributed Checkpoint (recommended for large models)
import torch.distributed.checkpoint as dcp

# Save
state_dict = {'model': fsdp_model.state_dict(), 'optim': optimizer.state_dict()}
dcp.save(state_dict, checkpoint_id='./checkpoints/step_1000')

# Load
state_dict = {'model': fsdp_model.state_dict(), 'optim': optimizer.state_dict()}
dcp.load(state_dict, checkpoint_id='./checkpoints/step_1000')
fsdp_model.load_state_dict(state_dict['model'])
optimizer.load_state_dict(state_dict['optim'])

```

4. Tensor Parallelism (TP)

Tensor Parallelism splits individual weight matrices across GPUs. Unlike data parallelism (which duplicates the model), TP distributes each layer horizontally. This is essential when a single layer's weights exceed GPU memory, or when scaling beyond 256 GPUs where AllReduce ring latency dominates.

4.1 Column-wise and Row-wise Sharding

TP for a linear layer $Y = XW^T$ follows the Megatron-LM pattern. The weight matrix W can be split either column-wise (output features partitioned) or row-wise (input features partitioned). These two splits are typically paired: column-wise first, row-wise second, so that the intermediate AllReduce is avoided.

Megatron-LM Splitting Pattern

MLP block: $Y = \text{GeLU}(XA)B$

- A is split column-wise (each GPU holds $A_i = A[:, i*k:(i+1)*k]$)
- B is split row-wise (each GPU holds $B_i = B[i*k:(i+1)*k, :]$)
- AllReduce after B_i to get full Y

Attention block: Q, K, V linear layers split column-wise; output projection split row-wise

- Each GPU handles $\text{num_heads} / N$ attention heads
- Single AllReduce per attention block

4.2 PyTorch Tensor Parallel API (DTensor)

```
# Tensor Parallelism using torch.distributed.tensor.parallel
import torch
import torch.nn as nn
from torch.distributed.device_mesh import init_device_mesh
from torch.distributed.tensor.parallel import (
    parallelize_module,
    ColwiseParallel,
    RowwiseParallel,
    PrepareModuleInput,
    SequenceParallel,
)

# — Initialize 1-D TP mesh (8 GPUs on one node) —
tp_mesh = init_device_mesh('cuda', (8,), mesh_dim_names=('tp',))

# — Simple MLP —
class MLP(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.fc1 = nn.Linear(dim, 4 * dim)
```

```

        self.act = nn.GELU()
        self.fc2 = nn.Linear(4 * dim, dim)
    def forward(self, x):
        return self.fc2(self.act(self.fc1(x)))

mlp = MLP(dim=1024).cuda()

# — Apply TP: fc1 column-wise, fc2 row-wise —
parallelize_module(
    module=mlp,
    device_mesh=tp_mesh,
    parallelize_plan={
        'fc1': ColwiseParallel(),    # split output features
        'fc2': RowwiseParallel(),    # split input features
    }
)

# — Forward pass — same API as normal module —
x = torch.randn(16, 1024, device='cuda')
y = mlp(x) # communication handled transparently

```

4.3 Sequence Parallelism

Sequence Parallelism (SP) extends TP by sharding along the sequence dimension in regions where TP does not shard (e.g., LayerNorm, Dropout). This further reduces activation memory during long-sequence training.

```

# Apply Sequence Parallelism to LayerNorm
from torch.distributed.tensor.parallel import SequenceParallel

parallelize_module(
    module=transformer_block,
    device_mesh=tp_mesh,
    parallelize_plan={
        'attn_norm': SequenceParallel(),    # LayerNorm sharded on seq dim
        'attn.wq': ColwiseParallel(),
        'attn.wk': ColwiseParallel(),
        'attn.wv': ColwiseParallel(),
        'attn.wo': RowwiseParallel(),
        'ffn_norm': SequenceParallel(),    # LayerNorm sharded on seq dim
        'ffn.w1': ColwiseParallel(),
        'ffn.w2': RowwiseParallel(),
    }
)

```


5. Pipeline Parallelism (PP)

Pipeline Parallelism splits the model vertically — consecutive layers (or groups of layers) are assigned to different GPUs. Data flows through stages like an assembly line. Multiple micro-batches are in flight simultaneously across different stages, reducing idle time ('pipeline bubble').

5.1 Pipeline Schedules

Schedule	Description
GPipe (synchronous)	All micro-batches forward, then all backward. Simple but has large pipeline bubble ($1 - 1/m$ for m micro-batches).
1F1B (PipeDream)	Each stage alternates 1 forward and 1 backward micro-batch. Reduces bubble and memory footprint.
Interleaved 1F1B	Megatron-LM's virtual pipeline: each GPU holds multiple non-consecutive stage chunks. Reduces bubble further at cost of more communication.
Zero-bubble (V-schedule)	Theoretical zero bubble using careful scheduling of B and W passes separately.

5.2 PyTorch Native Pipeline API

```
# torch.distributed.pipelining - PyTorch 2.2+
import torch
import torch.nn as nn
from torch.distributed.pipelining import pipeline, SplitPoint
from torch.distributed.pipelining import ScheduleGPipe, Schedule1F1B

# — Define model —
class Transformer(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(50257, 1024)
        self.layers = nn.ModuleList([TransformerBlock(1024) for _ in range(24)])
        self.head = nn.Linear(1024, 50257)
    def forward(self, x):
        x = self.embed(x)
        for layer in self.layers: x = layer(x)
        return self.head(x)

# — Split model into 4 pipeline stages —
# Method A: Manual split points
model = Transformer()
example_input = torch.randint(0, 50257, (4, 512)) # (batch, seq)

pipe = pipeline(
    module=model,
```

```

mb_args=(example_input,),
split_spec={
    'layers.5': SplitPoint.BEGINNING,    # Stage 0: embed + layers 0-4
    'layers.11': SplitPoint.BEGINNING,   # Stage 1: layers 5-10
    'layers.17': SplitPoint.BEGINNING,   # Stage 2: layers 11-16
    # Stage 3: layers 17-23 + head
}
)

# — Each process takes one stage —
stage_id = dist.get_rank()    # 4 ranks for 4 stages
stage_mod = pipe.get_stage_module(stage_id)

from torch.distributed.pipelining import PipelineStage
stage = PipelineStage(
    stage_mod,
    stage_id,
    num_stages=4,
    device=device,
)

# — Schedule and execute —
schedule = Schedule1F1B(stage, n_microbatches=8)

if stage_id == 0:            # first stage receives input
    schedule.step(example_input)
else:
    schedule.step()

```

5.3 Pipeline Best Practices

- **Micro-batch count:** Use 8–16 micro-batches per pipeline flush to reduce bubble ratio to < 6%.
- **Stage balance:** Ensure roughly equal computation across stages to avoid stragglers.
- **Memory:** 1F1B stores $O(\text{stages})$ activation buffers vs GPipe's $O(\text{micro-batches} \times \text{stages})$.
- **Communication:** Only activations at stage boundaries travel across GPUs — much less traffic than TP's AllReduce per layer.
- **Combine with DP/TP:** Standard practice: TP within a node (NVLink), PP across nodes, DP across replica groups — 3D parallelism.

6. Expert Parallelism (EP) and Mixture of Experts

Mixture-of-Experts (MoE) models use sparse activation: only a subset of 'expert' networks (usually feed-forward networks) process each token. Expert Parallelism distributes these experts across GPUs — each GPU owns a partition of experts. Tokens are dispatched via AllToAll, processed by local experts, then returned via another AllToAll.

6.1 MoE Architecture Overview

MoE Token Routing Pipeline

1. Gating / Router: For each token, a learned gating network produces probabilities over all E experts.
2. Top-K Selection: Each token selects the top-K experts (typically K=1 or K=2).
3. Dispatch (AllToAll): Tokens are sent to the GPUs that own their assigned experts.
4. Expert Computation: Each GPU runs its local experts on the received tokens.
5. Combine (AllToAll): Expert outputs are sent back to originating GPUs.
6. Weighted Sum: Outputs weighted by gating probabilities are combined.

Load-Balancing Loss: An auxiliary loss penalises router collapse (all tokens to one expert).

Z-Loss: Stabilises router probabilities from becoming too extreme.

6.2 Simple MoE Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Expert(nn.Module):
    def __init__(self, dim, ffn_dim):
        super().__init__()
        self.w1 = nn.Linear(dim, ffn_dim)
        self.w2 = nn.Linear(ffn_dim, dim)
    def forward(self, x): return self.w2(F.gelu(self.w1(x)))

class MoELayer(nn.Module):
    def __init__(self, dim, ffn_dim, num_experts=8, top_k=2):
        super().__init__()
        self.num_experts = num_experts
        self.top_k = top_k
        self.gate = nn.Linear(dim, num_experts, bias=False)
        self.experts = nn.ModuleList([Expert(dim, ffn_dim) for _ in
range(num_experts)])

    def forward(self, x):
        B, T, D = x.shape
        x_flat = x.view(-1, D) # (B*T, D)
        logits = self.gate(x_flat) # (B*T, num_experts)
        probs = F.softmax(logits, dim=-1)
```

```

# Top-K gating
topk_probs, topk_ids = probs.topk(self.top_k, dim=-1) # (B*T, K)
topk_probs = topk_probs / topk_probs.sum(dim=-1, keepdim=True)

# Dispatch tokens to experts
out = torch.zeros_like(x_flat)
for k in range(self.top_k):
    for e in range(self.num_experts):
        mask = (topk_ids[:, k] == e) # which tokens go to expert e
        if mask.any():
            expert_in = x_flat[mask]
            expert_out = self.experts[e](expert_in)
            out[mask] += topk_probs[mask, k:k+1] * expert_out

# Load balancing auxiliary loss
freq = probs.mean(dim=0) # avg prob per expert
ideal_freq = torch.ones_like(freq) / self.num_experts
balance_loss = (freq * probs.mean(dim=0)).sum() * self.num_experts

return out.view(B, T, D), balance_loss

# Usage
moe = MoELayer(dim=1024, ffn_dim=4096, num_experts=8, top_k=2)
x = torch.randn(2, 512, 1024)
y, aux_loss = moe(x)
total_loss = main_loss + 0.01 * aux_loss # auxiliary loss weight

```

6.3 Expert Parallelism with AllToAll

```

# Expert Parallelism - each GPU hosts num_experts // world_size experts
import torch.distributed as dist

class DistributedMoE(nn.Module):
    def __init__(self, dim, ffn_dim, total_experts, top_k=2):
        super().__init__()
        self.world_size = dist.get_world_size()
        self.rank = dist.get_rank()
        self.total_experts = total_experts
        self.local_experts = total_experts // self.world_size
        self.top_k = top_k
        self.gate = nn.Linear(dim, total_experts, bias=False)
        # Each GPU only instantiates its local partition of experts
        self.experts = nn.ModuleList([
            Expert(dim, ffn_dim) for _ in range(self.local_experts)
        ])

    def forward(self, x):
        B, T, D = x.shape
        tokens = x.view(-1, D) # (N, D) where N = B*T
        logits = self.gate(tokens)
        _, topk_ids = logits.topk(self.top_k, dim=-1)

```

```

# Build send/recv counts for AllToAll
# ... (token bucketing and sort by expert id)

# AllToAll dispatch: send tokens to owning GPUs
# dist.all_to_all sends each GPU's tokens to the correct rank
dispatched = [None] * self.world_size
dist.all_to_all(dispatched, local_expert_tokens_list)

# Run local experts on received tokens
local_out = [self.experts[i](dispatched[i]) for i in
range(self.local_experts)]

# AllToAll combine: send results back
combined = [None] * self.world_size
dist.all_to_all(combined, local_out)

# Merge weighted outputs and return
return output.view(B, T, D)

```

6.4 MoE with PyTorch DTensor (Production Pattern)

```

# PyTorch blog pattern - Expert Parallelism via DTensor + DeviceMesh
from torch.distributed.device_mesh import init_device_mesh
from torch.distributed._tensor import DTensor, Shard, Replicate

# 2D mesh: (data_parallel, expert_parallel)
# e.g., 32 GPUs = 4 DP × 8 EP
mesh_2d = init_device_mesh('cuda', (4, 8),
                           mesh_dim_names=('dp', 'ep'))
ep_mesh = mesh_2d['ep']    # expert parallel sub-mesh
dp_mesh = mesh_2d['dp']    # data parallel sub-mesh

# Expert weights sharded along expert dimension
# Each of the 8 EP ranks holds E/8 experts
# AllToAll dispatch/combine is handled via DTensor collectives

```

7. DeviceMesh, DTensor & 3D / 4D Parallelism

7.1 DeviceMesh

DeviceMesh (`torch.distributed.device_mesh`) is a multi-dimensional abstraction for organising distributed devices. A 2-D mesh of shape (dp, tp) means dp data-parallel replicas each split across tp tensor-parallel GPUs. This allows composable parallelism without manually managing process groups.

```
from torch.distributed.device_mesh import init_device_mesh

# 1-D mesh (all 8 GPUs in one group — for DDP/FSDP)
mesh_1d = init_device_mesh('cuda', (8,))

# 2-D mesh: 2 DP groups × 4 TP GPUs = 8 GPUs total
mesh_2d = init_device_mesh('cuda', (2, 4),
                           mesh_dim_names=('dp', 'tp'))
dp_mesh = mesh_2d['dp'] # ProcessGroup for data parallelism
tp_mesh = mesh_2d['tp'] # ProcessGroup for tensor parallelism

# 3-D mesh: PP × DP × TP
# 64 GPUs: 2 pipeline stages × 4 DP replicas × 8 TP
mesh_3d = init_device_mesh('cuda', (2, 4, 8),
                           mesh_dim_names=('pp', 'dp', 'tp'))
```

7.2 DTensor

DTensor (`torch.distributed._tensor`) is a tensor subclass that carries sharding metadata (placement) alongside the local tensor data. Placements can be `Shard(dim)` — the tensor is partitioned along a given dimension — or `Replicate()` — the tensor is identically held on all ranks.

```
from torch.distributed._tensor import DTensor, Shard, Replicate
from torch.distributed.device_mesh import init_device_mesh

mesh = init_device_mesh('cuda', (4,))

# Distribute a tensor column-wise (shard on dim 1)
weight = torch.randn(1024, 4096) # full weight on all ranks initially
dt = DTensor.from_local(weight.chunk(4, dim=1)[dist.get_rank()],
                       mesh, [Shard(1)])

# Convert back to full tensor (AllGather)
full = dt.full_tensor() # each rank now has the full tensor

# Replicated DTensor
bias = torch.zeros(1024)
```

```
dt_bias = DTensor.from_local(bias, mesh, [Replicate()])
```

7.3 3D Parallelism: TP + FSDP + PP

For training models with 10B+ parameters, combining all three parallelisms — Tensor Parallelism (intra-node), FSDP (data parallelism + ZeRO sharding), and Pipeline Parallelism (inter-node) — is the standard industry approach. This is called 3D parallelism.

```
# 3D Parallelism: TP (8) + FSDP (4) + PP (2) = 64 GPUs total
# Reference: TorchTitan (github.com/pytorch/torchtitan)

from torch.distributed.device_mesh import init_device_mesh
from torch.distributed.fsdp import fully_shard
from torch.distributed.tensor.parallel import parallelize_module, ColwiseParallel,
RowwiseParallel
from torch.distributed.pipeline import pipeline, ScheduleF1B, PipelineStage

# — Step 1: Build device mesh —
# world_size = 64 = 2 (PP) × 4 (FSDP/DP) × 8 (TP)
mesh_3d = init_device_mesh(
    'cuda', (2, 4, 8), mesh_dim_names=('pp', 'dp', 'tp')
)
pp_mesh = mesh_3d['pp']
dp_mesh = mesh_3d['dp']
tp_mesh = mesh_3d['tp']

# — Step 2: Apply Tensor Parallelism to each transformer block —
for block in model.layers:
    parallelize_module(block, tp_mesh, {
        'attn.wq': ColwiseParallel(),
        'attn.wv': ColwiseParallel(),
        'attn.wk': ColwiseParallel(),
        'attn.wo': RowwiseParallel(),
        'ffn.w1': ColwiseParallel(),
        'ffn.w2': RowwiseParallel(),
    })

# — Step 3: Apply FSDP2 to shard across DP dimension —
for block in model.layers:
    fully_shard(block, mesh=dp_mesh)
fully_shard(model, mesh=dp_mesh)

# — Step 4: Apply Pipeline Parallelism across PP dimension —
# Split model into 2 pipeline stages
pipe = pipeline(model, mb_args=(example_input,),
    split_spec={'layers.11': SplitPoint.BEGINNING})
stage_id = pp_mesh.get_local_rank()
stage = PipelineStage(pipe.get_stage_module(stage_id),
    stage_id, 2, device=device)
schedule = ScheduleF1B(stage, n_microbatches=8)
```


8. Mixed Precision Training

Mixed precision training uses lower-precision formats (fp16 or bf16) for the majority of computations to reduce memory and increase throughput on Tensor Core hardware, while maintaining fp32 for numerically sensitive operations like loss scaling and optimizer state updates.

8.1 fp16 vs. bf16

Format	Bits	Dynamic Range	Recommended for
fp32	32	Very large	Optimizer states, master weights
fp16	16 (1+5+10)	Moderate (± 65504)	Forward pass on Volta/Turing GPUs; needs loss scaling
bf16	16 (1+8+7)	Same as fp32	Ampere+ GPUs (A100, H100); no loss scaling needed
fp8 (H100)	8	Very small	H100 Transformer Engine; experimental quantisation

8.2 Automatic Mixed Precision (torch.amp)

```
import torch
from torch.cuda.amp import autocast, GradScaler

model      = MyModel().cuda()
optimizer  = torch.optim.AdamW(model.parameters(), lr=1e-4)
scaler     = GradScaler()          # only needed for fp16, not bf16

for x, y in dataloader:
    x, y = x.cuda(), y.cuda()
    optimizer.zero_grad()

    # — fp16 path —
    with autocast(device_type='cuda', dtype=torch.float16):
        out = model(x)
        loss = criterion(out, y)

    scaler.scale(loss).backward()          # scaled loss to prevent underflow
    scaler.unscale_(optimizer)             # unscale before clipping
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)                 # skips step if inf/nan detected
    scaler.update()                        # adjusts scale factor

    # — bf16 path (simpler — no scaler needed) —
```

```
with autocast(device_type='cuda', dtype=torch.bfloat16):
    out = model(x)
    loss = criterion(out, y)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
optimizer.zero_grad()
```

8.3 Mixed Precision with DDP

```
# AMP + DDP: just combine - they compose naturally
ddp_model = DDP(model.cuda(), device_ids=[local_rank])
scaler    = GradScaler()

for x, y in loader:
    optimizer.zero_grad()
    with autocast(device_type='cuda', dtype=torch.float16):
        loss = criterion(ddp_model(x.cuda()), y.cuda())
    scaler.scale(loss).backward() # gradient AllReduce still happens here
    scaler.step(optimizer)
    scaler.update()
```

9. Training Optimisations

9.1 Gradient Checkpointing (Activation Checkpointing)

Gradient (activation) checkpointing trades compute for memory. During the forward pass, intermediate activations are NOT saved. During the backward pass, they are recomputed on-the-fly. This reduces activation memory from $O(\text{layers})$ to $O(\sqrt{\text{layers}})$ at the cost of ~33% extra compute.

```
from torch.utils.checkpoint import checkpoint, checkpoint_sequential

# — Method 1: checkpoint a function —
class CheckpointedBlock(nn.Module):
    def __init__(self): super().__init__(); self.block = TransformerBlock(1024)
    def forward(self, x):
        return checkpoint(self.block, x, use_reentrant=False)
    # use_reentrant=False is recommended for PyTorch 2.x

# — Method 2: checkpoint_sequential for nn.Sequential —
seq_model = nn.Sequential(*[TransformerBlock(1024) for _ in range(24)])

def forward(x):
    return checkpoint_sequential(seq_model, segments=6, input=x)
    # Splits into 6 segments; activations only saved at segment boundaries

# — Method 3: Per-module via __init__ —
class EfficientTransformer(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.ModuleList([TransformerBlock(1024) for _ in range(24)])
        self.use_ckpt = True

    def forward(self, x):
        for layer in self.layers:
            if self.use_ckpt:
                x = checkpoint(layer, x, use_reentrant=False)
            else:
                x = layer(x)
        return x
```

9.2 Gradient Accumulation

Gradient accumulation simulates a larger effective batch size by running multiple forward-backward passes before calling `optimizer.step()`. This is essential when per-GPU memory limits the physical batch size.

```
accum_steps = 8    # effective batch = batch_size * accum_steps * world_size
optimizer.zero_grad()
```

```

for step, (x, y) in enumerate(loader):
    x, y = x.to(device), y.to(device)

    with autocast(device_type='cuda', dtype=torch.bfloat16):
        loss = criterion(model(x), y) / accum_steps # normalise

    loss.backward()

    if (step + 1) % accum_steps == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()

```

9.3 Efficient Data Loading

```

from torch.utils.data import DataLoader

# Performance-optimised DataLoader settings
loader = DataLoader(
    dataset,
    batch_size=32,
    sampler=DistributedSampler(dataset),
    num_workers=4,          # multiple CPU workers for prefetching
    pin_memory=True,        # pinned CPU memory for fast GPU transfers
    prefetch_factor=2,      # batches to prefetch per worker
    persistent_workers=True, # keep workers alive between epochs
    drop_last=True,         # avoid unequal last batch in distributed setting
)

# Profile data loading bottleneck
import time
t0 = time.perf_counter()
for batch in loader: pass
print(f'Data loading time: {time.perf_counter()-t0:.2f}s')

```

9.4 torch.compile

`torch.compile` (PyTorch 2.0+) uses TorchInductor to JIT-compile model operations into optimised CUDA kernels, typically delivering 20–50% speedup with minimal code changes.

```

import torch

model = MyModel().cuda()

# — Basic compilation —
model = torch.compile(model)          # default mode ('default')

# — Modes —

```

```

model = torch.compile(model, mode='default')          # balanced compile/runtime
tradeoff
model = torch.compile(model, mode='reduce-overhead') # minimise Python overhead
model = torch.compile(model, mode='max-autotune')     # maximum kernel optimisation
(slow compile)

# — With DDP —
model      = torch.compile(model)    # compile BEFORE DDP wrapping
ddp_model = DDP(model, device_ids=[local_rank])

# — With FSDP2 —
fully_shard(model, mesh=mesh)        # shard first
model = torch.compile(model)         # compile after

# — Dynamic shapes (for variable-length sequences) —
model = torch.compile(model, dynamic=True)

# — Inspect compiled graph —
explained = torch._dynamo.explain(model)(dummy_input)
print(explained.graphs)              # view captured computation graphs

```

9.5 torch.profiler — Identifying Bottlenecks

```

from torch.profiler import profile, record_function, ProfilerActivity

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True,
    profile_memory=True,
    with_stack=True,
    schedule=torch.profiler.schedule(
        wait=1, warmup=1, active=3, repeat=2
    ),
    on_trace_ready=torch.profiler.tensorboard_trace_handler('./log/prof')
) as prof:
    for step, (x, y) in enumerate(loader):
        with record_function('forward'):
            out = model(x.cuda())
        with record_function('backward'):
            criterion(out, y.cuda()).backward()
        prof.step()

print(prof.key_averages().table(sort_by='cuda_time_total', row_limit=20))
# Launch TensorBoard: tensorboard --logdir=./log/prof

```

9.6 Flash Attention & Fused Kernels

```

# Flash Attention 2 – memory-efficient attention (O(N) instead of O(N^2))
# PyTorch 2.0+ has native support via F.scaled_dot_product_attention
import torch.nn.functional as F

```

```

def efficient_attention(q, k, v, mask=None):
    # Automatically uses Flash Attention or memory-efficient attention
    return F.scaled_dot_product_attention(
        q, k, v,
        attn_mask=mask,
        dropout_p=0.0,
        is_causal=True,    # causal masking for autoregressive models
    )

# Check which backend is selected
with torch.backends.cuda.sdp_kernel(
    enable_flash=True,
    enable_math=False,
    enable_mem_efficient=True,
):
    out = efficient_attention(q, k, v)

# Fused optimizer (reduces kernel launch overhead)
optimizer = torch.optim.AdamW(
    model.parameters(), lr=1e-4,
    fused=True,          # uses fused CUDA kernel — requires PyTorch 2.0+
)

```

9.7 Optimisation Checklist

1. Set NCCL_DEBUG=INFO to diagnose communication issues.
2. Use CUDA_VISIBLE_DEVICES to control GPU visibility per process.
3. Set OMP_NUM_THREADS=1 to avoid CPU over-subscription.
4. Enable torch.backends.cudnn.benchmark = True for fixed-size inputs.
5. Use pin_memory=True and persistent_workers=True in DataLoader.
6. Apply torch.compile() with mode='max-autotune' for production inference.
7. Use bfloat16 on Ampere+ GPUs — no loss scaling required, same dynamic range as fp32.
8. Profile with torch.profiler before optimising — identify actual bottlenecks.
9. Use gradient_as_bucket_view=True in DDP to reduce memory copies.
10. Set PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to reduce fragmentation.

10. Multi-GPU Fine-tuning

10.1 Fine-tuning Strategies Overview

Strategy	Description
Full fine-tuning	Update all parameters. Best quality but highest compute/memory cost.
LoRA / QLoRA	Low-Rank Adaptation: insert trainable rank-r matrices; freeze base weights. ~3–10× less memory.
Prefix Tuning / Prompt Tuning	Prepend trainable token embeddings; freeze all model weights.
Adapters	Insert small trainable modules between frozen layers.
PEFT + FSDP	Apply FSDP to only the trainable LoRA parameters for memory efficiency.

10.2 Full Fine-tuning with FSDP

```
# Fine-tune LLaMA-7B on a custom dataset using FSDP
# torchrun --nproc_per_node=8 finetune_fsd.py

import os, functools
import torch
import torch.distributed as dist
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import ShardingStrategy, MixedPrecision
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers.models.llama.modeling_llama import LlamaDecoderLayer

dist.init_process_group(backend='nccl')
local_rank = int(os.environ['LOCAL_RANK'])
torch.cuda.set_device(local_rank)

# Load model on CPU (critical for memory efficiency)
model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    torch_dtype=torch.bfloat16,
    low_cpu_mem_usage=True,      # avoids double-loading
)

# FSDP wrap policy - one shard per decoder layer
wrap_policy = functools.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={LlamaDecoderLayer},
)

fsdp_model = FSDP(
    model,
    auto_wrap_policy=wrap_policy,
```

```

        mixed_precision=MixedPrecision(
            param_dtype=torch.bfloat16,
            reduce_dtype=torch.float32,
        ),
        sharding_strategy=ShardingStrategy.FULL_SHARD,
        device_id=torch.cuda.current_device(),
        use_orig_params=True,
    )

    # Apply activation checkpointing to reduce activation memory
    from torch.distributed.algorithms._checkpoint.checkpoint_wrapper import (
        apply_activation_checkpointing, CheckpointWrapper
    )
    apply_activation_checkpointing(
        fsdp_model,
        check_fn=lambda m: isinstance(m, LlamaDecoderLayer),
    )

    optimizer = torch.optim.AdamW(
        fsdp_model.parameters(), lr=2e-5, weight_decay=0.01
    )
    tokenizer = AutoTokenizer.from_pretrained('meta-llama/Llama-2-7b-hf')

    # Training loop (abbreviated)
    for batch in dataloader:
        input_ids = batch['input_ids'].to(local_rank)
        labels    = batch['labels'].to(local_rank)
        optimizer.zero_grad()
        outputs    = fsdp_model(input_ids=input_ids, labels=labels)
        outputs.loss.backward()
        fsdp_model.clip_grad_norm_(1.0)
        optimizer.step()

```

10.3 LoRA Fine-tuning with PEFT + FSDP

```

# pip install peft transformers
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM

# — Configure LoRA —
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,                                     # rank (smaller = less memory)
    lora_alpha=32,                             # scaling factor (alpha/r = effective learning
rate scale)
    lora_dropout=0.1,
    target_modules=['q_proj', 'v_proj', 'k_proj', 'o_proj',
                   'gate_proj', 'up_proj', 'down_proj'],
    bias='none',
)

base_model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',

```



```

    torch_dtype=torch.bfloat16,
    low_cpu_mem_usage=True,
)

# — Wrap with LoRA —
model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# Expected: trainable params ~0.1% of total

# — Apply FSDP only to trainable LoRA parameters —
# Freeze base model weights – only LoRA adapters are updated
for name, param in model.named_parameters():
    if 'lora' not in name:
        param.requires_grad_(False)

# Wrap with FSDP (will shard LoRA params + frozen base across GPUs)
from torch.distributed.fsdcp import FullyShardedDataParallel as FSDP
fsdp_model = FSDP(
    model,
    auto_wrap_policy=wrap_policy,
    mixed_precision=mp_policy,
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    device_id=local_rank,
    use_orig_params=True,
)

# Save LoRA adapter only
model.save_pretrained('lora_adapter/') # saves only ~25 MB vs 14 GB full model

# Load & merge later
from peft import PeftModel
merged = PeftModel.from_pretrained(base_model, 'lora_adapter/')
merged = merged.merge_and_unload() # merge weights for inference

```

10.4 QLoRA — Quantised Fine-tuning

```

# QLoRA: quantise base model to 4-bit, train LoRA adapters in bf16
# pip install bitsandbytes transformers peft accelerate
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import prepare_model_for_kbit_training, LoraConfig, get_peft_model

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4', # Normal Float 4-bit
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True, # quantise quantisation constants
)

model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-13b-hf',
    quantization_config=bnb_config,
    device_map='auto', # auto-distributes layers across GPUs
)

```

```
)

model = prepare_model_for_kbit_training(model) # enables gradient checkpointing

config = LoraConfig(
    r=64, lora_alpha=128,
    target_modules=['q_proj', 'v_proj'],
    lora_dropout=0.05,
    bias='none',
    task_type='CAUSAL_LM',
)
model = get_peft_model(model, config)

# QLoRA trains a 13B model in < 12 GB GPU memory
```

11. Elastic Training & torchrun

11.1 Elastic Training Concepts

Elastic training (`torch.distributed.elastic`) allows a distributed job to change its world size (add or remove workers) during training, without restarting from scratch. This is valuable for fault-tolerant training on cloud spot instances.

Concept	Description
Rendezvous	Dynamic registration mechanism where workers find each other. Backends: C10d (default), etcd, torchx.
min/max_nodes	Minimum and maximum number of nodes the job can run with.
Elastic agent	Process supervisor on each node that monitors workers and triggers re-sync on failure/addition.
State dict	Application must save/load checkpoints to resume from elastic events.

11.2 torchrun Elastic Launch

```
# — Elastic launch: tolerate 1-4 nodes —
torchrun \
  --nproc_per_node=8 \
  --nnodes=1:4 \           # min:max nodes
  --rdzv_backend=c10d \
  --rdzv_endpoint=${MASTER_ADDR}:29400 \
  --rdzv_id=my_job_001 \
  --max_restarts=3 \       # restart attempts before failing
  train_elastic.py

# — SLURM integration —
#!/bin/bash
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=1
#SBATCH --gpus-per-task=8

nodes=($(scontrol show hostnames $SLURM_JOB_NODELIST))
head_node=${nodes[0]}
head_ip=$(srun --nodes=1 --ntasks=1 -w $head_node hostname --ip-address)

srun torchrun \
  --nnodes=$SLURM_NNODES \
  --nproc_per_node=8 \
  --rdzv_backend=c10d \
  --rdzv_endpoint=${head_ip}:29500 \
  train.py
```

11.3 Fault-Tolerant Training Pattern

```

# Elastic-compatible training loop
import torch.distributed as dist
from torch.distributed.elastic.multiprocessing.errors import record

@record # captures and reports worker errors
def train():
    dist.init_process_group(backend='nccl')
    rank = dist.get_rank()

    model = MyModel().to(device)
    ddp = DDP(model, device_ids=[local_rank])

    start_epoch = 0
    # Load checkpoint if exists (critical for elastic resume)
    ckpt_path = f'./ckpt/latest.pt'
    if os.path.exists(ckpt_path):
        ckpt = torch.load(ckpt_path, map_location='cpu')
        model.load_state_dict(ckpt['model'])
        optimizer.load_state_dict(ckpt['optim'])
        start_epoch = ckpt['epoch'] + 1

    for epoch in range(start_epoch, max_epochs):
        # ... training loop ...
        if rank == 0 and epoch % save_every == 0:
            torch.save({
                'model': ddp.module.state_dict(),
                'optim': optimizer.state_dict(),
                'epoch': epoch,
            }, ckpt_path)
            dist.barrier() # ensure all ranks wait for checkpoint

    dist.destroy_process_group()

if __name__ == '__main__':
    train()

```

12. Debugging & Monitoring

12.1 Common Issues & Fixes

Issue	Fix
Training hangs / NCCL timeout	Check all ranks call identical collectives. Use NCCL_DEBUG=INFO. Ensure firewalls allow NCCL ports.
CUDA out-of-memory (OOM)	Reduce batch size, enable gradient checkpointing, use bf16/fp16, switch from DDP to FSDP.
find_unused_parameters hang	Set find_unused_parameters=True or ensure all params participate in loss.
Non-deterministic results	Set torch.manual_seed(seed) and sampler.set_epoch(epoch) on all ranks.
Slow training (bottleneck)	Profile with torch.profiler. Check GPU utilisation with nvidia-smi or nsight.
Unequal batch sizes (last batch)	Use drop_last=True in DataLoader or handle padding explicitly.
FSDP checkpoint load failure	Ensure SHARDED_STATE_DICT loaded with same world_size, or use FULL_STATE_DICT.
Gradient explosion / NaN loss	Add gradient clipping (clip_grad_norm_). Use bf16 instead of fp16 (no scaling needed).

12.2 Useful Environment Variables

```
# NCCL debugging
export NCCL_DEBUG=INFO           # verbose NCCL logging
export NCCL_DEBUG_SUBSYS=ALL     # all subsystems
export NCCL_SOCKET_IFNAME=eth0   # force specific network interface
export NCCL_IB_DISABLE=1        # disable InfiniBand (use TCP)

# CUDA debugging
export CUDA_LAUNCH_BLOCKING=1    # synchronous CUDA launches (for error tracing)
export TORCH_SHOW_CPP_STACKTRACES=1

# Memory
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True

# Performance
export OMP_NUM_THREADS=1         # avoid CPU thread over-subscription
export CUDA_VISIBLE_DEVICES=0,1,2,3 # control GPU visibility

# Distributed backend
export MASTER_ADDR=localhost
export MASTER_PORT=29500
export RANK=0
```

```
export WORLD_SIZE=4
export LOCAL_RANK=0
```

12.3 Distributed Monitoring with torch.distributed

```
import torch.distributed as dist
import torch.distributed.distributed_c10d as c10d

# Check if distributed is available and initialised
print(dist.is_available())      # True if torch was built with dist support
print(dist.is_initialized())    # True after init_process_group

# Monitor gradient norms for debugging
total_norm = 0.0
for p in model.parameters():
    if p.grad is not None:
        total_norm += p.grad.data.norm(2).item() ** 2
total_norm = total_norm ** 0.5
if dist.get_rank() == 0:
    print(f'Grad norm: {total_norm:.4f}')

# Check parameter sync across ranks
def check_params_sync(model):
    for name, param in model.named_parameters():
        tensor = param.data.clone()
        dist.broadcast(tensor, src=0)
        if not torch.allclose(param.data, tensor, atol=1e-6):
            print(f'DESYNC detected in {name}!')
            return False
    return True
```

13. End-to-End Production Workflow

13.1 Decision Guide: Choosing Your Strategy

Strategy Selection Flow

Does your model fit on a single GPU with target batch size?

- └ YES → Use DDP (simplest, best throughput for medium models)
- └ NO →

Does the model fit if you use bf16 + gradient checkpointing?

- └ YES → Use DDP + AMP + gradient checkpointing
- └ NO →

Do you have > 8 GPUs or multi-node?

- └ YES, model is 1B–70B params → Use FSDP (ZeRO-3)
- └ YES, model is 70B+ params → Use FSDP + Tensor Parallelism (2D)
- └ YES, model is 1T+ params → Use 3D: TP + FSDP + PP

Using MoE architecture? → Add Expert Parallelism to any of the above.

Want fast fine-tuning? → LoRA + FSDP or QLoRA (single GPU possible).

Need fault tolerance? → Use torchrun elastic + distributed checkpointing.

13.2 Complete Training Script Template

```
#!/usr/bin/env python3
# complete_train.py
# torchrun --nproc_per_node=8 complete_train.py --config config.yaml

import os, argparse, yaml, time, functools
import torch
import torch.nn as nn
import torch.distributed as dist
import torch.distributed.checkpoint as dcp
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import ShardingStrategy, MixedPrecision
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from torch.cuda.amp import autocast
from torch.utils.data import DataLoader, DistributedSampler
from torch.optim.lr_scheduler import CosineAnnealingLR

# -----
# 1. Setup
# -----

def setup_distributed():
    dist.init_process_group(backend="nccl")
    local_rank = int(os.environ['LOCAL_RANK'])
    torch.cuda.set_device(local_rank)
    return dist.get_rank(), dist.get_world_size(), local_rank
```

```

# -----
# 2. Model + wrapper (DDP or FSDP based on config)
# -----
def build_model(config, local_rank):
    model = MyTransformer(**config['model'])

    if config['parallelism'] == 'ddp':
        model = model.to(local_rank)
        model = DDP(model, device_ids=[local_rank],
                     gradient_as_bucket_view=True)

    elif config['parallelism'] == 'fsdp':
        from TransformerBlock import TransformerBlock
        wrap_policy = functools.partial(
            transformer_auto_wrap_policy,
            transformer_layer_cls={TransformerBlock}
        )
        mp = MixedPrecision(param_dtype=torch.bfloat16,
                           reduce_dtype=torch.float32)
        model = FSDP(model, auto_wrap_policy=wrap_policy,
                     mixed_precision=mp,
                     sharding_strategy=ShardingStrategy.FULL_SHARD,
                     device_id=local_rank, use_orig_params=True)

    return model

# -----
# 3. Training loop
# -----
def train(config):
    rank, world_size, local_rank = setup_distributed()
    device = torch.device(f'cuda:{local_rank}')
    is_main = (rank == 0)

    model = build_model(config, local_rank)
    optimizer = torch.optim.AdamW(model.parameters(),
                                   lr=config['lr'],
                                   weight_decay=0.01,
                                   fused=True)
    scheduler = CosineAnnealingLR(optimizer, T_max=config['epochs'])

    dataset = MyDataset(config['data_path'])
    sampler = DistributedSampler(dataset)
    loader = DataLoader(dataset, batch_size=config['batch_size'],
                       sampler=sampler, num_workers=4,
                       pin_memory=True, drop_last=True)

    # Resume from checkpoint
    start_epoch = 0
    if os.path.isdir(config['checkpoint_dir']):
        sd = {'model': model.state_dict(), 'optimizer': optimizer.state_dict()}
        dcp.load(sd, checkpoint_id=config['checkpoint_dir'])
        model.load_state_dict(sd['model'])
        optimizer.load_state_dict(sd['optimizer'])

```



```

start_epoch = sd.get('epoch', 0) + 1

for epoch in range(start_epoch, config['epochs']):
    sampler.set_epoch(epoch)
    model.train()
    t0 = time.time()

    for step, (x, y) in enumerate(loader):
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()

        with autocast(device_type='cuda', dtype=torch.bfloat16):
            loss = nn.CrossEntropyLoss()(model(x), y)

        loss.backward()

        if config['parallelism'] == 'fsdp':
            model.clip_grad_norm_(1.0)
        else:
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)

        optimizer.step()

        if is_main and step % 50 == 0:
            tput = config['batch_size'] * world_size / (time.time() - t0)
            print(f'Ep {epoch} Step {step}: loss={loss.item():.4f}
throughput={tput:.1f} samp/s')
            t0 = time.time()

    scheduler.step()

    # Save distributed checkpoint every epoch
    if epoch % config['save_every'] == 0:
        sd = {'model': model.state_dict(),
            'optimizer': optimizer.state_dict(),
            'epoch': epoch}
        dcp.save(sd, checkpoint_id=f"{config['checkpoint_dir']}/epoch_{epoch}")
        if is_main: print(f'Checkpoint saved at epoch {epoch}')

    dist.destroy_process_group()

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True)
    args = parser.parse_args()
    config = yaml.safe_load(open(args.config))
    train(config)

```

14. Quick Reference

14.1 Collective Operations Cheat Sheet

Operation	Semantics and Use Case
<code>dist.all_reduce(t, op=SUM)</code>	Sum tensor <code>t</code> across all ranks; result on all ranks. DDP gradient sync.
<code>dist.reduce(t, dst=0)</code>	Sum tensor <code>t</code> ; result only on rank 0. Loss aggregation.
<code>dist.broadcast(t, src=0)</code>	Rank 0 sends <code>t</code> to all ranks. Model init, weight broadcast.
<code>dist.all_gather(ts, t)</code>	Each rank contributes <code>t</code> ; all ranks get list <code>ts</code> of all contributions. FSDP AllGather params.
<code>dist.gather(t, ts, dst=0)</code>	Each rank sends <code>t</code> to rank 0 who receives list <code>ts</code> . Evaluation aggregation.
<code>dist.scatter(t, ts, src=0)</code>	Rank 0 distributes list <code>ts</code> to all ranks. Data sharding.
<code>dist.reduce_scatter(out, in_list)</code>	Reduce across ranks and scatter shards. FSDP gradient ReduceScatter.
<code>dist.all_to_all(out_list, in_list)</code>	Each rank sends different data to each rank. Expert Parallelism token dispatch.
<code>dist.barrier()</code>	Synchronisation point — all ranks wait. Before checkpoint save.

14.2 DDP vs. FSDP Comparison

Aspect	DDP	FSDP (ZeRO-3)	FSDP Hybrid Shard
Memory per GPU	Full model copy	1/N of full model	1/N (intra-node)
Communication	AllReduce grads	AllGather + ReduceScatter	AllGather local + AllReduce global
Complexity	Low	Medium–High	High
When to use	< 8 GPUs, model fits	Model too large for DDP	> 1 node, bandwidth-limited
Checkpoint	Standard <code>state_dict</code>	Sharded or full <code>state_dict</code>	Sharded <code>state_dict</code>
<code>torch.compile</code> compat	Full	Full (FSDP2)	Partial

14.3 Parallelism Composition Reference

Combination	GPU Count	Use Case
DDP only	2–8	Standard multi-GPU; model fits 1 GPU
FSDP (ZeRO-3)	8–64	Models 1B–70B parameters

Combination	GPU Count	Use Case
FSDP + TP (2D)	64–256	Models 70B–200B
FSDP + TP + PP (3D)	256–4096	Models 200B+ (GPT-4 scale)
3D + EP (4D)	4096+	Sparse MoE with 200B+ params
DDP + AMP	2–8	Standard fine-tuning with memory savings
FSDP + LoRA	4–8	Memory-efficient large model fine-tuning
QLoRA (single GPU)	1	7B–13B fine-tuning on consumer hardware

14.4 Key APIs Summary

API	Module
DistributedDataParallel	<code>torch.nn.parallel.DistributedDataParallel</code>
FullyShardedDataParallel (FSDP1)	<code>torch.distributed.fsdp.FullyShardedDataParallel</code>
<code>fully_shard</code> (FSDP2)	<code>torch.distributed.fsdp.fully_shard</code>
<code>init_device_mesh</code>	<code>torch.distributed.device_mesh.init_device_mesh</code>
<code>parallelize_module</code> (TP)	<code>torch.distributed.tensor.parallel.parallelize_module</code>
DTensor	<code>torch.distributed._tensor.DTensor</code>
pipeline (PP)	<code>torch.distributed.pipelining.pipeline</code>
PipelineStage	<code>torch.distributed.pipelining.PipelineStage</code>
autocast (AMP)	<code>torch.cuda.amp.autocast</code>
GradScaler	<code>torch.cuda.amp.GradScaler</code>
checkpoint (AC)	<code>torch.utils.checkpoint.checkpoint</code>
Distributed Checkpoint	<code>torch.distributed.checkpoint</code>
<code>torch.compile</code>	<code>torch</code> (top-level)
<code>torch.profiler</code>	<code>torch.profiler.profile</code>
DistributedSampler	<code>torch.utils.data.distributed.DistributedSampler</code>

14.5 Recommended Resources

- **Official Tutorials:** docs.pytorch.org/tutorials — DDP, FSDP, TP, PP, MoE cookbooks
- **TorchTitan:** github.com/pytorch/torchtitan — Reference 3D parallel LLaMA training
- **FSDP2 Migration Guide:** docs.pytorch.org/docs/stable/distributed.fsdp.fully_shard.html

- **Megatron-LM:** github.com/NVIDIA/Megatron-LM — TP/PP/SP for large LLMs
- **DeepSpeed:** deepspeed.ai — ZeRO optimiser, pipeline, MoE, offloading
- **PEFT Library:** github.com/huggingface/peft — LoRA, QLoRA, prefix tuning
- **PyTorch Distributed Blog:** pytorch.org/blog — Release notes, MoE training, FSDP2